BACKEND ARCHITECTURE

FINAL E-COMMERCE BACKEND ARCHITECTURE DOCUMENT

Node.js + Express.js + JavaScript + MongoDB
Modular Monolith + Practical DDD + Event-Driven Architecture + Transactional Outbox

Purpose: Production-oriented backend blueprint for a multi-vendor e-commerce platform with **USER, SELLER and ADMIN** panels. There is **no internal delivery panel**; shipment, pickup and tracking are handled through third-party logistics providers such as Shiprocket.

Adaptation basis: This document follows the uploaded food-backend architecture's structure and principles, while replacing food/restaurant/delivery-partner concepts with e-commerce seller/catalog/cart/order/shipping concepts.

1. Final Executive Decision

Use a **Modular Monolith** instead of microservices. Keep one repository and coordinated application while separating business capabilities into independent modules.

The final architecture is: **Modular Monolith + Practical DDD + Event-Driven Architecture + Transactional Outbox + MongoDB + Redis + BullMQ + Socket.IO**.

DDD should be practical. Strong domain modelling is required for complex areas such as Order, Payment, Inventory, Pricing, Coupons and Shipping. Simple CRUD modules should remain simple.

Seller, user and admin are application roles/panels within the same backend; they are not separate microservices.

2. Final Technology Stack

Runtime: Node.js — asynchronous API and background workloads.

Framework: Express.js — HTTP API layer.

Language: JavaScript — project requirement.

Database: MongoDB — primary persistent data store.

ODM: Mongoose — schemas, validation and indexes.

Cache: Redis — caching and temporary distributed state.

Background jobs: BullMQ — queues, retries, delayed jobs and workers.

Realtime: Socket.IO — order, payment and shipment-status updates where realtime UX is required.

Validation: Zod — validate request data before business logic.

Authentication: JWT access tokens + refresh sessions.

Logging: Pino — structured production logs.

Security: Helmet + CORS + rate limiting + validation.

Deployment: Docker + Nginx + horizontally scalable API instances.

3. High-Level Architecture

USER PANEL | SELLER PANEL | ADMIN PANEL
↓
HTTPS
↓
Nginx / Load Balancer
↓
Multiple Express API Instances
↓
Modular Monolith
↓
MongoDB + Redis + Socket.IO
↓
BullMQ Workers
↓
Payment / Shipping / SMS / Email / Push / Storage / Maps Providers

4. What Each Architecture Part Does

Modular Monolith: keeps one codebase but prevents the project from becoming one large unstructured codebase.

DDD: keeps important business rules close to the domain instead of scattering them across controllers.

Event-Driven: modules react to business events without tightly coupling all side effects to one request.

Transactional Outbox: stores critical events reliably with the database change.

Third-party provider abstraction: keeps Shiprocket/payment/notification/storage SDK-specific code outside business logic.

5. Final Folder Structure

```
backend/
■■■ src/
■ ■■■ app/
■ ■ ■■■ app.js
■ ■ ■■■ server.js
■ ■ ■■■ routes.js
■ ■ ■■■ container.js
■ ■■■ config/
■ ■ ■■■ env.js
■ ■ ■■■ database.js
■ ■ ■■■ redis.js
■ ■ ■■■ logger.js
■ ■ ■■■ constants.js
■ ■■■ modules/
■ ■ ■■■ identity/
■ ■ ■■■ user/
■ ■ ■■■ seller/
■ ■ ■■■ catalog/
■ ■ ■■■ category/
■ ■ ■■■ product/
■ ■ ■■■ inventory/
■ ■ ■■■ cart/
■ ■ ■■■ wishlist/
■ ■ ■■■ address/
■ ■ ■■■ order/
■ ■ ■■■ payment/
■ ■ ■■■ shipping/
■ ■ ■■■ coupon/
■ ■ ■■■ review/
■ ■ ■■■ notification/
■ ■ ■■■ admin/
■ ■ ■■■ search/
■ ■ ■■■ analytics/
■ ■■■ shared/
■ ■ ■■■ errors/
■ ■ ■■■ middleware/
■ ■ ■■■ utils/
■ ■ ■■■ constants/
■ ■ ■■■ validators/
■ ■■■ infrastructure/
■ ■ ■■■ cache/
■ ■ ■■■ queue/
■ ■ ■■■ events/
■ ■ ■■■ realtime/
■ ■ ■■■ storage/
■ ■ ■■■ external/
■ ■ ■■■ payment/
■ ■ ■■■ shipping/
■ ■ ■■■ notification/
■ ■ ■■■ maps/
■ ■■■ workers/
■ ■ ■■■ notification.worker.js
■ ■ ■■■ payment.worker.js
■ ■ ■■■ shipping.worker.js
■ ■ ■■■ analytics.worker.js
■ ■ ■■■ cleanup.worker.js
■ ■ ■■■ outbox.worker.js
■ ■■■ bootstrap/
■ ■■■ registerModules.js
```

```
■ ■■■ registerEvents.js
■ ■■■ registerWorkers.js
■ ■■■ gracefulShutdown.js
■■■ tests/
■■■ scripts/
■■■ docker/
■■■ .env
■■■ .env.example
■■■ package.json
■■■ README.md
```

6. Why app.js and server.js Are Separate

app.js: creates Express, registers middleware, routes and global error handling.

server.js: starts infrastructure: database, Redis, events, workers and HTTP server.

7. Standard DDD Module Structure

```
modules/order/
■■■ domain/
■ ■■■ entities/Order.js
■ ■■■ value-objects/
■ ■ ■■■ Money.js
■ ■ ■■■ OrderItem.js
■ ■■■ services/
■ ■ ■■■ OrderPricingService.js
■ ■ ■■■ OrderStatusService.js
■ ■■■ events/
■ ■ ■■■ OrderCreatedEvent.js
■ ■ ■■■ OrderConfirmedEvent.js
■ ■ ■■■ OrderCancelledEvent.js
■ ■ ■■■ OrderShippedEvent.js
■ ■■■ constants/orderStatus.js
■■■ application/
■ ■■■ use-cases/
■ ■ ■■■ CreateOrder.js
■ ■ ■■■ ConfirmOrder.js
■ ■ ■■■ CancelOrder.js
■ ■ ■■■ CompleteOrder.js
■ ■■■ dto/
■ ■■■ ports/OrderRepository.js
■■■ infrastructure/
■ ■■■ persistence/
■ ■ ■■■ OrderModel.js
■ ■ ■■■ MongoOrderRepository.js
■ ■■■ events/
■■■ presentation/
■■■ OrderController.js
■■■ OrderRoutes.js
■■■ OrderValidation.js
```

8. Layer Responsibilities

Presentation: routes, controllers and request validation. Controllers stay thin.

Application: use cases orchestrate workflows such as CreateOrder.

Domain: entities and business rules protect valid state transitions.

Infrastructure: MongoDB, Redis, queues and external provider implementations.

HTTP Request → **Presentation** → **Application / Use Case** → **Domain Rules** → **Repository / Provider Port** → **Infrastructure Implementation**

9. Final Business Modules

Identity: signup, login, OTP, password, JWT, refresh sessions and role-based access.

User: profile, preferences, addresses, wishlist and user settings.

Seller: seller onboarding, KYC/documents, store profile, store settings, staff, payout configuration and seller status.

Catalog: categories, brands, products, variants, attributes, images and product availability.

Inventory: stock, reservations, stock movements, low-stock rules and seller inventory.

Cart: add/update/remove products and checkout preparation.

Order: order creation, seller-wise order grouping, confirmation, cancellation, fulfillment and completion.

Payment: payment attempts, provider integration, webhooks, refunds and payment reconciliation.

Shipping: shipment creation, labels, AWB, tracking, pickup, delivery status and third-party provider webhooks.

Coupon: coupon rules, eligibility, usage limits and redemption records.

Review: product reviews, ratings, seller reviews and moderation.

Notification: push, SMS, email and in-app notifications.

Admin: seller approval, catalog moderation, order/payment/shipping monitoring, configuration and audit logs.

Search: product/category/brand search and filters.

Analytics: asynchronous business metrics and reports.

10. Event-Driven Architecture

Create Order
↓
Save Order + required state changes
↓
order.created
↓
Internal Event Dispatch
↓
Notification Handler | Inventory Handler | Payment Handler | Shipping Handler | Analytics Handler
↓
BullMQ
↓
Workers

Recommended business events: user.created, seller.approved, product.created, product.updated, inventory.updated, order.created, order.confirmed, order.cancelled, payment.succeeded, payment.failed, shipment.created, shipment.picked_up, shipment.in_transit, shipment.delivered, shipment.failed, refund.created, refund.completed and review.created.

11. Event Bus vs Queue

Event: communicates that a business fact happened.

BullMQ job: represents background work that may be slow or retryable.

Worker: performs the job.

Outbox: makes important event publication recoverable.

12. Transactional Outbox Pattern

MongoDB Transaction
■■■ Save Order / Inventory State
■■■ Save Outbox Event (PENDING)
↓
Commit
↓
Outbox Worker
↓
Dispatch Event
↓
Mark as Processed

This prevents a common failure scenario where an order or inventory change is saved successfully but the server crashes before notifications, shipping or other event processing is triggered.

Recommended fields: eventId, eventType, aggregateType, aggregateId, payload, status, attempts, availableAt, createdAt and processedAt.

13. Idempotency

Assume jobs, webhooks and events can be retried. A consumer should safely ignore or handle duplicates.

Use event IDs, idempotency keys and unique constraints for sensitive flows such as payments, refunds, inventory reservation and shipping creation.

14. Complete E-Commerce Order Creation Flow

POST /api/v1/orders
↓
Authentication
↓
Authorization
↓
Validation
↓
Order Controller
↓
CreateOrder Use Case
↓
Validate Cart
↓
Validate Seller/Product/Variant
↓
Check Inventory / Reserve Stock
↓
Calculate Price Server-Side
↓
Apply Coupon Rules
↓
Calculate Tax / Shipping Charges
↓
MongoDB Transaction
■■■ Create Order
■■■ Create Inventory Reservation/Movement

■■■ Create Outbox Event
↓
Commit
↓
Return Response
↓
Background: Outbox → Event → Handler → BullMQ → Worker

Never trust product prices, taxes, discounts, shipping charges or final totals received from the client. The server must calculate authoritative totals.

15. Order State Management

CREATED → PAYMENT_PENDING → PAID → CONFIRMED → PROCESSING → PACKED → SHIPPED → OUT_FOR_DELIVERY → DELIVERED

Explicit alternate paths must exist for CANCELLED, PAYMENT_FAILED, REFUND_PENDING, REFUNDED, SHIPMENT_FAILED and RETURN/REPLACEMENT flows where the business supports them.

A central status-transition rule must reject invalid transitions. Seller actions must be authorized only for orders belonging to that seller.

16. Order Snapshot Design

Store historical snapshots of seller, product and pricing information used at the time of purchase. Do not rely only on current catalog data.

```
{
orderNumber,
seller: { id, storeName },
items: [
{ productId, variantId, name, sku, price, quantity, selectedOptions }
],
pricing: {
subtotal, discount, tax, shippingFee, total
},
shippingAddress: { ... }
}
```

17. MongoDB Collections

users, userSessions

sellers, sellerDocuments, sellerStaff

categories, brands, products, productVariants

inventory, inventoryMovements, stockReservations

carts, wishlists

addresses

orders, orderItems, orderEvents

payments, paymentAttempts, refunds

shipments, shipmentEvents, shippingLabels, trackingEvents

notifications

coupons, couponUsages

reviews

outboxEvents, processedEvents

auditLogs

analyticsSnapshots

18. MongoDB Index Strategy

users: unique email/phone where applicable.

sellers: status + createdAt; searchable seller identifiers; geospatial index only if store/location queries actually require it.

products: sellerId + status; categoryId + status; SKU/variant identifiers; indexes based on real search/filter patterns.

inventory: sellerId + product/variant identifier; low-stock query patterns where required.

orders: userId + createdAt; sellerId + status + createdAt; status + createdAt; payment/order identifiers where applicable.

shipments: orderId; tracking/AWB identifier; status + updatedAt.

payments: unique provider payment/order identifiers.

Do not index every field. Design compound indexes from actual query and sorting patterns.

19. Redis Strategy

Cache product details, categories, seller/store data and read-heavy home/search feeds.

Store OTPs with TTL.

Distributed rate limiting.

Temporary locks and idempotency keys.

Socket.IO scaling support.

BullMQ backing storage.

Short-lived inventory reservation state only where the business design requires it; MongoDB remains the source of truth for durable inventory.

20. Cache Pattern

Request → Redis Lookup → HIT: Return | MISS: MongoDB → Store Cache → Return

Invalidate relevant keys when source data changes. For example, product.updated or inventory.updated should invalidate affected product/cache keys.

Do not cache authoritative payment, order or inventory truth in a way that can bypass MongoDB consistency rules.

21. Authentication and Authorization

Use short-lived access tokens plus longer-lived refresh sessions.

Store session metadata and hashed refresh-token data where applicable.

Support device logout and logout from all devices.

Roles: USER, SELLER, SELLER_STAFF (if required), ADMIN and SUPER_ADMIN.

Authorization must combine role checks with ownership checks.

Seller users must only access their own catalog, inventory, orders, shipments, reports and store configuration unless an explicit admin permission allows otherwise.

22. Payment Architecture

Create Order / Payment Attempt
↓
Payment Provider
↓
Client Payment
↓
Webhook
↓
Verify Signature
↓
Persist Result
↓
payment.succeeded / payment.failed
↓
Order + Inventory + Notification + Analytics

Do not treat a frontend success screen as the final payment source of truth. Verify provider webhooks and make webhook processing idempotent.

23. Third-Party Provider Abstraction

```
infrastructure/external/
■■■ payment/
■ ■■■ PaymentProvider.js
■ ■■■ RazorpayProvider.js
■■■ shipping/
■ ■■■ ShippingProvider.js
■ ■■■ ShiprocketProvider.js
■■■ notification/
■ ■■■ PushProvider.js
■ ■■■ SmsProvider.js
■ ■■■ EmailProvider.js
■■■ storage/
■ ■■■ StorageProvider.js
■■■ maps/
■■■ MapsProvider.js
```

Application modules depend on provider abstractions rather than scattering vendor SDK calls throughout the codebase.

Shipping integration should support create shipment, generate/fetch label, pickup request, tracking, cancellation where supported, and webhook status synchronization.

Shiprocket is an example provider; the architecture should allow another logistics provider to be added without rewriting Order or Seller business logic.

24. Shipping and Realtime Architecture — No Delivery Panel

Seller/Admin/User
↓
Backend Shipping Module
↓
ShippingProvider abstraction
↓

Shiprocket / Other Logistics Provider
↓
Courier / Delivery Network
↓
Third-Party Tracking Webhook
↓
Backend Webhook Handler
↓
Shipment + Order Status Update
↓
Socket.IO / Notification

There is no DELIVERY_PARTNER role or delivery app in this platform. The backend stores shipment/tracking information received from the third-party logistics provider.

Do not poll the logistics provider excessively. Prefer provider webhooks where available; use scheduled reconciliation jobs only as a fallback for stale or missed statuses.

25. BullMQ Worker Strategy

notification.worker.js — push/in-app notification work.

payment.worker.js — payment reconciliation or asynchronous payment-related jobs where required.

shipping.worker.js — shipment creation, label retrieval, tracking reconciliation and provider retries.

analytics.worker.js — asynchronous metrics.

outbox.worker.js — dispatch pending reliable events.

cleanup.worker.js — scheduled cleanup.

Workers may run as separate processes while remaining part of the same codebase. This allows independent scaling without introducing microservices.

26. API Standards

/api/v1/auth

/api/v1/users

/api/v1/sellers

/api/v1/catalog

/api/v1/categories

/api/v1/products

/api/v1/inventory

/api/v1/cart

/api/v1/wishlist

/api/v1/addresses

/api/v1/orders

/api/v1/payments

/api/v1/shipments

/api/v1/coupons

/api/v1/reviews

/api/v1/notifications

/api/v1/admin

/api/v1/search

/api/v1/analytics

Success: { success: true, message, data, meta }

Error: { success: false, error: { code, message } }

27. Validation and Errors

Validate body, params and query before entering business logic.

Use Zod for request validation.

Centralize errors with AppError-style classes.

Typical errors: ValidationError, UnauthorizedError, ForbiddenError, NotFoundError, ConflictError, InsufficientStockError, PaymentError and ExternalServiceError.

Never expose internal stack traces or secrets in production responses.

28. Logging and Monitoring

Use Pino structured logging.

Attach requestId to requests and logs.

Track API latency, error rate, MongoDB latency, Redis health, queue depth, worker failures, payment webhook failures and Socket connections.

Track shipping provider API latency/errors, webhook lag, shipment creation failures and tracking synchronization failures.

Use error tracking and infrastructure monitoring in production.

29. Security Checklist

Helmet.

Strict CORS.

Rate limiting.

Request validation.

Password hashing.

Short-lived access tokens.

Refresh session management.

Request body limits.

Webhook signature verification for payment and shipping providers.

Environment-based secrets.

Admin audit logs.

Role and ownership authorization.

Seller isolation: seller A must never access seller B's catalog, inventory, order or financial data.

30. Request Lifecycle

CLIENT
↓
NGINX / LOAD BALANCER
↓
EXPRESS
↓
Request ID + Logging
↓
Security
↓
Rate Limit
↓
Authentication
↓
Authorization
↓
Validation
↓
Controller
↓
Use Case
↓
Domain Rules
↓
Repository / Provider
↓
MongoDB / Redis / External Provider
↓
Outbox when required
↓
STANDARD RESPONSE

31. Scaling Principles for 50K+ Users

Keep API instances stateless.

Do not keep critical shared application state in Node process memory.

Use MongoDB and Redis for shared state.

Scale API instances horizontally behind a load balancer.

Scale workers independently.

Cache read-heavy catalog data.

Design indexes from production query patterns.

Load-test checkout, inventory reservation, payment webhooks and shipment flows.

Monitor actual bottlenecks before introducing additional complexity.

Registered-user count alone does not define server requirements. Concurrent users, requests per second, peak checkout/order volume, database load and realtime connections are important capacity metrics.

32. Strict Dependency Rules

Presentation → Application → Domain.

Infrastructure provides implementations.

Need immediate data → controlled module interface/query.

Need asynchronous side effect → event.

Avoid uncontrolled direct access to another module's database models.

Seller, product, inventory and order modules must communicate through defined interfaces and events instead of arbitrary cross-module model access.

Not every interaction should become an event; use synchronous module interfaces when immediate consistent information is genuinely required.

33. Recommended Packages

express, mongoose, ioredis, bullmq, socket.io, jsonwebtoken, bcrypt OR argon2, zod, helmet, cors, compression, pino, pino-http, dotenv, uuid, http-status-codes, express-rate-limit, rate-limit-redis.

Testing: jest OR vitest, supertest.

Documentation: swagger-jsdoc, swagger-ui-express.

Provider SDKs should be added only when actually required by the chosen payment, shipping, notification or storage provider.

34. Team Implementation Rules

Keep controllers thin.

Keep business rules out of random utility files.

Calculate critical pricing, tax, discounts and shipping charges on the server.

Never trust client-side inventory or price values.

Do not scatter third-party SDK calls.

Do not use database transactions for every CRUD operation; use them where atomicity is actually required.

Do not rely on exactly-once event delivery.

Make critical handlers idempotent.

Do not add indexes blindly.

Do not overengineer simple CRUD with unnecessary DDD patterns.

Keep each module responsible for its own business capability.

Do not install unnecessary dependency packages.

Prefer centralized reusable services/utilities when the same responsibility can be safely shared.

Keep APIs optimized: return only required fields, paginate large lists, avoid unbounded queries and avoid unnecessary populate/join-like expansion.

Do not introduce microservices unless real traffic, deployment independence or team boundaries later prove a specific module needs extraction.

35. Recommended Implementation Sequence

1. Foundation: config, app, logging, errors.
2. MongoDB + Redis + middleware.
3. Identity + sessions + authorization.
4. User + Seller onboarding + seller approval.
5. Catalog + Category + Product + Variant.
6. Inventory + reservation rules.
7. Cart + Wishlist + Address.
8. Order domain + pricing + state rules + outbox.
9. Payment + webhook verification + refund flow.
10. Shipping integration + Shiprocket provider + shipping webhooks.
11. Notifications + BullMQ workers.
12. Review + Coupon + Search.
13. Admin + analytics + audit logs.
14. Testing + load testing + monitoring + deployment.

36. Final E-Commerce Architecture Blueprint

MODULAR MONOLITH
+
PRACTICAL DDD
+
EVENT-DRIVEN ARCHITECTURE
+
TRANSACTIONAL OUTBOX
+
NODE.JS + EXPRESS + JAVASCRIPT
+
MONGODB + MONGOOSE
+
REDIS
+
BULLMQ WORKERS
+
SOCKET.IO
+
THIRD-PARTY SHIPPING PROVIDER (e.g. SHIPROCKET)

Final recommendation: Start with this architecture as one disciplined backend. Build clean module boundaries now, use reliable events for asynchronous side effects, isolate seller data correctly, make payment/shipping webhooks idempotent, and scale API/worker processes independently. Keep shipping provider-specific logic behind an abstraction so the platform does not become coupled to one logistics vendor.